Reimbursement Management System

Design decisions, trade-offs and viva preparation

MIS number	Name
612415001	Abdurrahman Shaikh
612415057	Dhruv Agarwal
612415044	Mrudula Dagale
612415093	Makarand S Kulkarni

DBMS Laboratory · October 2026

Contents

1. How to use this document **3**
1.1 The design in one page 3
2. The final E-R diagram **4**
2.1 Notation key 5
2.2 Every relationship, with its justification 5
2.3 Specializations 7
2.4 What changed since the ER assignment 7
3. ER modelling decisions **8**
4. Reduction to relation schemas **14**
4.1 Rules applied 14
4.2 The relation schemas 14
4.3 Mapping decisions 15
5. Functional dependencies and normalization **19**
5.1 Dependencies that hold 19
5.2 Dependencies that do not hold 19
5.3 Dependencies designed out 20
5.4 Normal form of every relation 20
5.5 Walkthrough from the unnormalized claim form 20
6. Constraint and implementation decisions **22**
6.1 Catalogue of named constraints 25
7. Testing **26**
8. Known limitations **26**
9. Viva preparation **27**
9.1 ER modelling 27
9.2 From ER to tables 28
9.3 Functional dependencies and normalization 29
9.4 Constraints and implementation 29
10. Demonstration script **30**

1. How to use this document

This document records every design decision in the database: what we chose, what else we considered, why, and what it costs. It is meant for the team to prepare for the viva, where the questions will be about database design.

- **Decision IDs.** E = ER modelling (Section 3), R = reduction to relations (Section 4), C = constraints and implementation (Section 6). The same IDs appear as comments in db/01_schema.sql and db/03_triggers.sql, e.g. "-- [E17]".
- **Textbook references** (§) are to Silberschatz, Korth & Sudarshan, Database System Concepts, 7th edition.
- **Proof that the rules work.** db/06_constraint_tests.sql runs 50 statements against the live database; each one either breaks a rule (and must be rejected) or sets up a scenario. All pass. Run it during the viva to demonstrate any rule.

1.1 The design in one page

The ER model. Employees belong to departments and report to supervisors. An employee may be an applicant (files claims), a manager (decides claims up to an approval limit) and an admin (pays claims), in any combination. An applicant submits reimbursements; each is exactly one of travel, medical or business, and contains one or more expense items (a weak entity). Managers review, and admins pay, the submission as a whole (an aggregation).

The schema. 11 relations, all in BCNF. One relation per entity set and subclass, one for the multivalued phone attribute; all relationships are many-to-one and merge into the many side.

The rules. 13 foreign keys, 4 unique keys, 18 CHECK constraints and 17 triggers. Keys and CHECKs handle everything that one row can show; triggers handle the rules that need other rows, the old value, or today's date.

2. The final E-R diagram

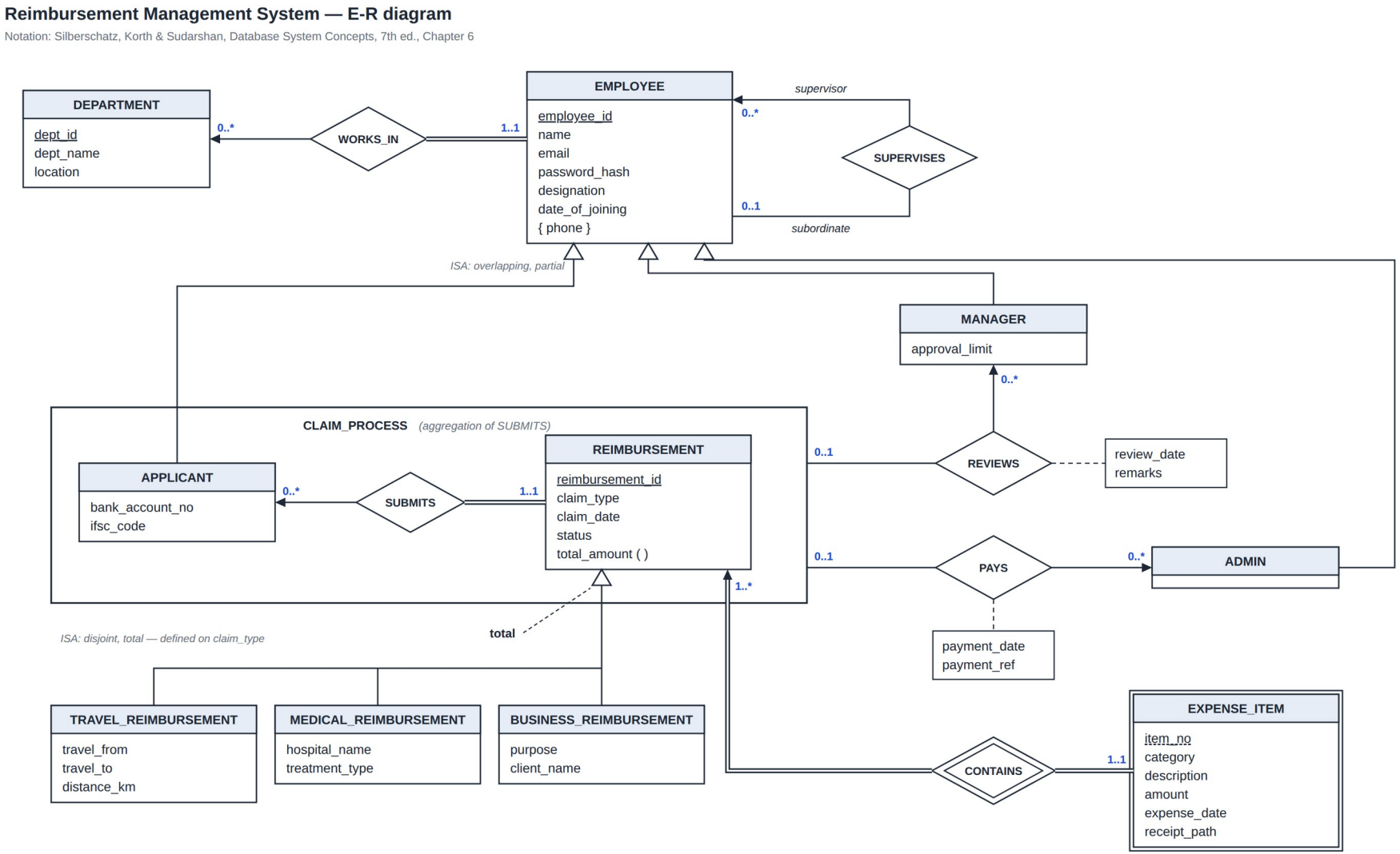


Figure 1. Final E-R diagram (textbook notation). Full-resolution copy with the key drawn in: report/er/er_diagram.png.

2.1 Notation key

Symbol in the diagram	Meaning (Silberschatz Chapter 6)
Rectangle with a shaded header	Entity set; attributes listed below the name; primary key underlined (§6.2)
Double rectangle	Weak entity set; its discriminator has a dashed underline (§6.5.3)
Diamond / double diamond	Relationship set / identifying relationship of a weak entity set
Arrow on a line (→)	The "one" side: each entity at the other end relates to at most one entity here (§6.4)
Plain line	The "many" side
Double line	Total participation: every entity of that set takes part in the relationship
l..h on a line	Minimum and maximum number of relationships each entity of that set takes part in (e.g. 1..1, 0..*)
Role label on a line (supervisor, subordinate)	Role indicator, needed when an entity set relates to itself
Rectangle linked by a dashed line to a diamond	Descriptive attributes of the relationship
{ phone }	Multivalued attribute
total_amount ( )	Derived attribute
Hollow arrowhead	Specialization (ISA), pointing to the superclass. Separate arrows = overlapping; one shared arrow = disjoint; the keyword "total" (dashed line to the arrowhead) = total specialization (§6.8)
Box around a relationship and its entity sets	Aggregation: the relationship treated as an entity set (§6.8.5)

2.2 Every relationship, with its justification

For each relationship: the mapping cardinality and participation we chose, how the diagram shows them (arrow, double line, l..h), what it becomes in the schema, and why. An l..h label describes the entity set it is drawn next to.

Relationship	Connects	Mapping cardinality	Participation	How it is drawn	Becomes in the schema	Why
WORKS_IN	EMPLOYEE – DEPARTMENT	Many-to-one	EMPLOYEE total; DEPARTMENT partial	Arrow to DEPARTMENT; double line from EMPLOYEE; 1..1 on EMPLOYEE, 0..* on DEPARTMENT	`employee.dept_id` NOT NULL, FK → department	Every employee belongs to exactly one department, because spend is reported by department. A new department may have nobody in it yet.
SUPERVISES	EMPLOYEE (supervisor) – EMPLOYEE (subordinate)	One-to-many	Both partial	Arrow on the supervisor line; role labels; 0..* (supervisor), 0..1 (subordinate)	`employee.supervisor_id` NULL, FK → employee	Each employee reports to at most one person. The head of the organisation reports to nobody, and most employees supervise nobody. Role indicators are needed because both ends are the same entity set.
SUBMITS	APPLICANT – REIMBURSEMENT	One-to-many	REIMBURSEMENT total; APPLICANT partial	Arrow to APPLICANT; double line from REIMBURSEMENT; 0..* on APPLICANT, 1..1 on REIMBURSEMENT	`reimbursement.applicant_id` NOT NULL, FK → applicant	A claim cannot exist without the person who filed it, and has exactly one claimant. A new applicant has filed nothing yet.
REVIEWS	MANAGER – CLAIM_PROCESS (aggregation)	One-to-many (one decision per claim)	Both partial	Arrow to MANAGER; plain line from the aggregation; 0..* on MANAGER, 0..1 on CLAIM_PROCESS; descriptive attributes review_date, remarks	`reimbursement.reviewed_by`, `review_date`, `remarks` (NULL until reviewed); reviewed_by FK → manager	A claim is decided once, by one manager (single-level approval). A submitted claim has no decision yet; a manager may have decided nothing.
PAYS	ADMIN – CLAIM_PROCESS (aggregation)	One-to-many (one payment per claim)	Both partial	Arrow to ADMIN; plain line from the aggregation; 0..* on ADMIN, 0..1 on CLAIM_PROCESS; descriptive attributes payment_date, payment_ref	`reimbursement.paid_by`, `payment_date`, `payment_ref` (NULL until paid); paid_by FK → admin	An approved claim is paid by one bank transfer, recorded by one admin. Claims that are not approved are never paid.
CONTAINS (identifying)	REIMBURSEMENT – EXPENSE_ITEM (weak)	One-to-many	Both total	Double diamond; arrow and double line to REIMBURSEMENT; double line to EXPENSE_ITEM; 1..* on REIMBURSEMENT, 1..1 on EXPENSE_ITEM	No relation of its own; expense_item has PK (reimbursement_id, item_no) and FK → reimbursement ON DELETE CASCADE	An item only exists inside its claim (existence dependency). A claim with no items has nothing to reimburse, so REIMBURSEMENT participates totally too.

2.3 Specializations

Specialization	Subclasses	Constraints	How it is drawn	Becomes in the schema	Why
ISA on EMPLOYEE	APPLICANT, MANAGER, ADMIN	Overlapping, partial	Three separate hollow arrows; no "total" keyword	Tables applicant, manager, admin; each PK is also an FK → employee	Roles combine: managers also file claims, and Sneha (finance) is both an applicant and an admin. An intern without a payout account (Karan) has no role yet.
ISA on REIMBURSEMENT	TRAVEL_, MEDICAL_, BUSINESS_ REIMBURSEMENT	Disjoint, total; attribute-defined on claim_type	One shared hollow arrow; "total" keyword joined by a dashed line	Tables travel_/medical_/business_ reimbursement; each PK is also an FK → reimbursement	Every claim is exactly one kind, and each kind has its own mandatory details and policy.

2.4 What changed since the ER assignment

Kept as it was	Changed	Added
EMPLOYEE and its overlapping specialization into APPLICANT and MANAGER SUPERVISES (recursive, 1:N) The aggregation CLAIM_PROCESS and SUBMITS EXPENSE_ITEM as a weak entity with CONTAINS Phone (multivalued), Total_Amount (derived) The three claim types	Department: attribute → entity set (E1) APPROVES → REVIEWS with attributes (E8) Bank details: + IFSC code (E6) Claim specialization qualified: disjoint, total (E10) REIMBURSEMENT participation in CONTAINS: partial → total (E13) Receipt → receipt_path (E15) Notation: ovals and 1/N → textbook notation	ADMIN subclass (E5) and PAYS (E9) password_hash (E16) category (E14) Status values and transitions (E11)

3. ER modelling decisions

One table per decision. "Enforced by" names the database objects that implement it.

E1 · DEPARTMENT is an entity set, not an attribute

Decision	Model DEPARTMENT (dept_id, dept_name, location) and link EMPLOYEE to it through WORKS_IN: many-to-one, EMPLOYEE total, DEPARTMENT partial.
Alternatives	Keep Department as a text attribute of EMPLOYEE, as in our ER assignment.
Why	A department has a fact of its own (location) and is shared by many employees. Textbook §6.9.2: use an entity set when the thing has attributes beyond its name or is referred to from several places. As an attribute, the name is repeated in every employee row (a typo silently creates a new department), and adding location would create employee_id → department → location: a transitive dependency that breaks 3NF.
Cost	One more table, and a join wherever the department name is shown.
Enforced by	`department` table; `employee.dept_id` NOT NULL with a RESTRICT foreign key; UNIQUE `dept_name`.

E2 · Phone is a multivalued attribute

Decision	Keep { phone } as a multivalued attribute of EMPLOYEE; it becomes `employee_phone (employee_id, phone)`.
Alternatives	Columns phone1, phone2; a comma-separated list in one column; a PHONE entity set.
Why	An employee has any number of numbers (mobile, desk, home). Fixed columns cap the number and create NULLs. A list in one column is not atomic, so the relation would not be in 1NF (§7.8); the database could not validate, index or search individual numbers. A phone number has no attributes of its own, so an entity set adds nothing over the multivalued attribute.
Cost	A join (or GROUP_CONCAT) to display all numbers.
Enforced by	Primary key (employee_id, phone) prevents duplicates; `chk_phone_format`; ON DELETE CASCADE from employee.

E3 · SUPERVISES is a recursive relationship on EMPLOYEE

Decision	Keep SUPERVISES on EMPLOYEE, one-to-many with role indicators supervisor and subordinate, partial on both sides.
Alternatives	Put SUPERVISES on MANAGER only; store the supervisor as an attribute; derive reporting lines from REVIEWS.
Why	Reporting lines and approval authority are different facts. In our data Sneha (finance, an admin but not a manager) supervises Meera, and a team lead might supervise people without any approval authority. So any employee can supervise. Both ends are EMPLOYEE, so role indicators are required to say which end is which (§6.2). Partial: the head of the organisation has no supervisor, and most employees supervise nobody.
Cost	A foreign key cannot stop cycles (A supervises B supervises A); a trigger with a recursive query does (C10).

Enforced by	`employee.supervisor_id` (nullable FK); `trg_employee_bi`, `trg_employee_bu`.

E4 · EMPLOYEE specialization is overlapping and partial

Decision	APPLICANT, MANAGER and ADMIN specialize EMPLOYEE. Overlapping (three separate arrows) and partial (no "total" keyword).
Alternatives	Disjoint roles; a single role column on employee; a generic employee_role (employee_id, role) table.
Why	Roles combine. Rahul, Priya and Neha manage and also file claims; Sneha files claims and pays them. A disjoint specialization would forbid this. Partial: Karan, an intern without a registered bank account, has no role yet, and Meera is only an admin. Each subclass has its own attributes or relationships (bank details; approval_limit; PAYS). A role column could not hold those without NULLs, and could not be the target of a foreign key: `reimbursement.reviewed_by` must point at a manager, not at any employee.
Cost	Three tables; finding a user's roles needs joins (the `employee_roles` view does them once).
Enforced by	Tables applicant, manager, admin (PK = FK to employee); foreign keys from reimbursement to the right subclass.

E5 · New ADMIN subclass with no attributes

Decision	Add ADMIN as a third subclass of EMPLOYEE. It has no attributes of its own.
Alternatives	An is_admin flag; let managers pay claims; keep admins outside EMPLOYEE in a separate users table.
Why	Someone must pay approved claims and maintain master data, and our ER assignment had no one to do it. Textbook §6.8.1: a subclass is justified when it has attributes or takes part in relationships that the superclass does not. ADMIN takes part in PAYS, so it is a proper subclass even with no attributes. A separate role for payment makes separation of duties possible: the person who approves a claim cannot pay it (C6). Admins are employees (they also file claims), so they belong inside the hierarchy, not in a separate table.
Cost	A one-column table.
Enforced by	`admin` table; `reimbursement.paid_by` references `admin`.

E6 · Bank details: IFSC code added; (IFSC, account number) is a candidate key

Decision	APPLICANT stores bank_account_no and ifsc_code, and the pair is UNIQUE.
Alternatives	The account number alone (our ER assignment); a BANK_ACCOUNT entity set; also storing bank name and branch.

Why	Indian account numbers are unique only within one bank, and NEFT/IMPS transfers need the IFSC code. Making the pair unique stops two employees from routing their payouts to the same account. Bank name and branch are determined by the IFSC code. Storing them would add ifsc_code → bank_name, whose left side is not a key of applicant: a 3NF violation. We do not store them; if needed they belong in a bank_branch (ifsc_code, bank_name, branch) relation.
Cost	One payout account per employee, which is enough here.
Enforced by	`uq_applicant_bank`; `chk_applicant_account` (9 to 18 digits); `chk_applicant_ifsc` (case-sensitive pattern).

E7 · Aggregation CLAIM_PROCESS is kept

Decision	REVIEWS and PAYS relate MANAGER and ADMIN to the aggregation of APPLICANT – SUBMITS – REIMBURSEMENT.
Alternatives	(a) Ternary relationships REVIEWS (MANAGER, APPLICANT, REIMBURSEMENT) and PAYS (ADMIN, APPLICANT, REIMBURSEMENT). (b) Binary relationships from MANAGER and ADMIN to REIMBURSEMENT only.
Why	(a) is the case the textbook uses to motivate aggregation (§6.8.5). The applicant would be recorded twice, once in SUBMITS and once in REVIEWS, and nothing would stop a review whose applicant differs from the person who submitted the claim. (b) gives exactly the same tables here, because SUBMITS is many-to-one: the aggregation's key is just reimbursement_id (§6.8.6.2). We keep the aggregation because it states what is actually decided and paid: a submission, meaning who claimed what. Two rules are about that pair: a manager may not review their own claim, and the money goes to the submitter's bank account.
Cost	None in the tables (the aggregation has no relation of its own); the diagram has one more box.
Viva note	Be ready to say that (b) would be relationally equivalent, and why we still prefer the aggregation.

E8 · APPROVES renamed REVIEWS, with review_date and remarks

Decision	One relationship REVIEWS records the decision, whatever it is; the outcome goes in the claim's status. One decision per claim.
Alternatives	Keep APPROVES and add a REJECTS relationship; multi-level approval (several reviews per claim).
Why	A manager may reject a claim, and "APPROVES" is the wrong name for a rejection. One relationship plus a status covers both outcomes with one set of columns, and keeps "decided at most once" easy to enforce. review_date is needed for the audit trail and for the rule review_date ≥ claim_date; remarks explain rejections. Multi-level approval would make REVIEWS many-to-many with its own relation (claim, level, manager, date, decision). We left it out of scope.
Cost	Approvals and rejections share columns, so the meaning of remarks depends on the status.
Enforced by	`reviewed_by`, `review_date`, `remarks`; `chk_reimbursement_lifecycle` (a rejection needs remarks).

E9 · New PAYS relationship

Decision	ADMIN pays CLAIM_PROCESS: one-to-many, both partial, with payment_date and payment_ref (the bank's UTR number).
Alternatives	A "paid" flag on the claim; payment details as extra attributes of REVIEWS.
Why	Payment is a separate event by a different person at a different time, with its own attributes. A unique payment_ref links each claim to one bank transfer and stops one transfer from being recorded twice. Partial on the claim side: only approved claims are ever paid.
Enforced by	`paid_by` (FK → admin), `payment_date`, UNIQUE `payment_ref`; lifecycle CHECK; trigger allows APPROVED → PAID only.

E10 · REIMBURSEMENT specialization is disjoint, total and attribute-defined

Decision	claim_type (TRAVEL, MEDICAL or BUSINESS) decides the subclass; every claim belongs to exactly one.
Alternatives	Overlapping (a claim with both travel and business parts); partial (generic claims with no type); one wide table holding every type's columns.
Why	Each type follows its own policy and has its own mandatory details. A claim that was two types at once would make it unclear which policy applies, so an employee files two claims instead. Total: there is no "other" kind of claim; the business type covers miscellaneous business costs. Attribute-defined: the subclass is visible in the superclass itself, so lists and reports can show the type without joining three tables.
Cost	MySQL cannot declare totality or disjointness across tables, so triggers do it (R8, C8).
Enforced by	`chk_reimbursement_type`; `trg_travel_bi`, `trg_medical_bi`, `trg_business_bi`; subclass primary keys; the review gate in `trg_reimbursement_bu`.

E11 · Status attribute with a fixed lifecycle

Decision	status ∈ {SUBMITTED, APPROVED, REJECTED, PAID}. Allowed moves: SUBMITTED → APPROVED or REJECTED, APPROVED → PAID. REJECTED and PAID are final.
Alternatives	Derive the status entirely from other columns (a decision flag plus whether reviewed_by and paid_by are NULL); add DRAFT and CANCELLED states; keep a status-history table.
Why	The approve-or-reject outcome has to be stored somewhere; one status column is simple to query and to index (the review and payment queues filter on it). Part of it is derivable (PAID exactly when paid_by is not NULL). That is a controlled redundancy: the lifecycle CHECK makes every inconsistent combination impossible, so no update anomaly can occur. This is a rule across columns of one row, not a functional dependency, so it does not affect the normal form. Withdrawn claims are deleted rather than marked CANCELLED. That is simpler, but the system keeps no record of withdrawals. A history table is out of scope; the dates and people in the row are the audit trail.
Enforced by	`chk_reimbursement_status`, `chk_reimbursement_lifecycle`, transitions in `trg_reimbursement_bu`, `trg_reimbursement_bd`.

E12 · Total_Amount is derived, never stored

Decision	The total of a claim is computed from its items by the function `claim_total()` and the view `reimbursement_summary`.
Alternatives	A stored total column kept up to date by triggers on expense_item, or by the application.
Why	A stored total repeats information already in the items. Every insert, update and delete of an item would have to update it, and any path that forgot would leave the two disagreeing: a classic update anomaly. Claims have a handful of items, so summing them is cheap. Items are frozen once a claim is reviewed (C9), so the total a manager approved cannot change before payment.
Cost	An aggregate in every query that shows totals; the view does one GROUP BY for all claims, not one per claim.
Textbook	§6.3 (derived attributes) and §6.7.2: they are not stored, but can be represented by functions.

E13 · EXPENSE_ITEM is a weak entity set; both sides of CONTAINS are total

Decision	EXPENSE_ITEM is identified by its claim plus the discriminator item_no (1, 2, 3 … within each claim). CONTAINS is the identifying relationship. REIMBURSEMENT also participates totally: at least one item per claim.
Alternatives	A strong entity with its own item_id; items as a multivalued composite attribute of REIMBURSEMENT.
Why	An item has no identity outside its claim ("item 2 of claim 17"), and deleting a claim must delete its items: an existence dependency (§6.5.3). A surrogate item_id would also work, but it would add a meaningless key and allow an item to be moved to a different claim. A multivalued composite attribute maps to the same table. A weak entity set is the clearer model when the thing has several attributes and a partial key of its own. A claim with no items has nothing to reimburse, hence 1..* on the REIMBURSEMENT side.
Cost	The "at least one item" rule cannot be a declarative constraint in MySQL (C8).
Enforced by	PK (reimbursement_id, item_no); FK ON DELETE CASCADE; `trg_expense_item_bd` (keep at least one); review gate; `trg_expense_item_bu` (identity cannot change).

E14 · Category is an attribute with a fixed set of values

Decision	category is an attribute of EXPENSE_ITEM, limited to nine values by a CHECK constraint.
Alternatives	An EXPENSE_CATEGORY entity set (a lookup table referenced by a foreign key).
Why	A category has no attributes of its own today, so a fixed list keeps the design simple.
Cost	Adding a category means changing the CHECK constraint (DDL). If categories gain attributes (a spending cap per category, an accounting code) or admins need to add them themselves, category should become an entity set; then adding one is an ordinary INSERT.

E15 · Receipts are files; the database stores their path

Decision	Uploaded receipts are saved on disk under random names; expense_item.receipt_path stores the name. It is optional.
Alternatives	Store the file in a BLOB column.
Why	Receipts are images and PDFs of up to a few megabytes. In the database they would bloat backups and the buffer pool and slow every query that touches expense_item. Small expenses (an auto-rickshaw fare) often have no receipt, so the column is nullable.
Cost	The files and the database can drift apart (for example, a file deleted by hand). The application deletes a claim's files when the claim is withdrawn.

E16 · Passwords are stored only as bcrypt hashes

Decision	employee.password_hash CHAR(60) holds a bcrypt hash (salted, deliberately slow).
Alternatives	Plain text; a fast hash such as SHA-256; a separate credentials table.
Why	Plain text exposes every password to anyone who reads the table. Fast hashes can be brute-forced quickly. bcrypt includes a salt and a cost factor. Every user is an employee, so the hash belongs with the employee. A separate one-to-one credentials table would be justified if non-employees (auditors, say) needed accounts.

E17 · The approval limit is per manager and gates every decision

Decision	Each manager has an approval_limit. A manager may approve or reject a claim only if its total is within that limit. Any eligible manager may decide; it need not be the applicant's supervisor, though the queue lists direct reports first.
Alternatives	Limits set per designation; only the applicant's supervisor may decide; the limit applies to approvals only.
Why	Per-manager limits are flexible: two team leads can have different authority. If limits came from the designation, storing them per manager would create designation → approval_limit, a transitive dependency (3NF violation). A DESIGNATION entity set would then be needed. Routing by limit rather than by supervisor means a claim never gets stuck because the applicant's own supervisor has too low a limit, and it keeps SUPERVISES (who reports to whom) separate from REVIEWS (who may decide what). Deciding a claim of a given size is the authority, whether the answer is yes or no, so the limit also applies to rejections.
Cost	A claim larger than every manager's limit cannot be reviewed. Admins must keep at least one limit above the largest claim (the director's limit is ₹5,00,000). Lowering a limit later does not affect decisions already made, which is correct, but the system keeps no history of limits.
Enforced by	`trg_reimbursement_bu` compares `claim_total()` with the reviewer's limit.

E18 · Distance belongs to the trip, not to the route

Decision	distance_km is the distance actually travelled on this trip, entered by the applicant.
Alternatives	A ROUTE (travel_from, travel_to, distance_km) entity set with fixed distances.

Why	Trips between the same cities differ (detours, return legs, local travel at the destination). So (travel_from, travel_to) → distance_km is not a functional dependency here, and travel_reimbursement is in BCNF.
Cost	If company policy fixed the reimbursable distance per route, that FD would hold, travel_reimbursement would violate 3NF, and distances would have to move to a ROUTE relation.

4. Reduction to relation schemas

4.1 Rules applied

ER construct	Rule (textbook)	Result
Strong entity sets DEPARTMENT, EMPLOYEE, REIMBURSEMENT	One relation with the same attributes; same primary key (§6.7.1)	department, employee, reimbursement
Multivalued attribute { phone }	New relation: owner's key + the attribute; both form the key (§6.7.2)	employee_phone (employee_id, phone)
Derived attribute total_amount ( )	Not stored; computed when needed (§6.7.2)	Function `claim_total()` and view `reimbursement_summary`
Weak entity set EXPENSE_ITEM	Owner's key + discriminator = primary key; FK to owner (§6.7.3)	expense_item (reimbursement_id, item_no, …)
Identifying relationship CONTAINS	Redundant; no relation (§6.7.5)	Represented by expense_item's foreign key
Many-to-one with total participation: WORKS_IN, SUBMITS	Merge into the "many" side (§6.7.6)	`employee.dept_id`, `reimbursement.applicant_id` (NOT NULL)
Many-to-one with partial participation: SUPERVISES, REVIEWS, PAYS	Merge into the "many" side, using NULLs (§6.7.6)	`supervisor_id`; `reviewed_by`, `review_date`, `remarks`; `paid_by`, `payment_date`, `payment_ref`
Specializations of EMPLOYEE and REIMBURSEMENT	Method 1: superclass relation plus one relation per subclass holding the superclass key (§6.8.6.1)	applicant, manager, admin; travel_/medical_/business_reimbursement
Aggregation CLAIM_PROCESS	No relation of its own; its key is the key of SUBMITS (§6.8.6.2)	Key reimbursement_id, so REVIEWS and PAYS merge into reimbursement

4.2 The relation schemas

1. department (dept_id, dept_name, location)
UNIQUE (dept_name)

2. employee (employee_id, name, email, password_hash, designation, date_of_joining, dept_id, supervisor_id)
UNIQUE (email); dept_id → department; supervisor_id → employee (NULL allowed)

3. employee_phone (employee_id, phone)
employee_id → employee

4. applicant (employee_id, bank_account_no, ifsc_code)
employee_id → employee; UNIQUE (ifsc_code, bank_account_no)

5. manager (employee_id, approval_limit)
employee_id → employee

6. admin (employee_id)
employee_id → employee

7. reimbursement (reimbursement_id, claim_type, applicant_id, claim_date, status, reviewed_by, review_date, remarks, paid_by, payment_date, payment_ref)
applicant_id → applicant; reviewed_by → manager (NULL allowed); paid_by → admin (NULL allowed); UNIQUE (payment_ref)

8. travel_reimbursement (reimbursement_id, travel_from, travel_to, distance_km)
reimbursement_id → reimbursement

9. medical_reimbursement (reimbursement_id, hospital_name, treatment_type)
reimbursement_id → reimbursement

10. business_reimbursement (reimbursement_id, purpose, client_name)
reimbursement_id → reimbursement

11. expense_item (reimbursement_id, item_no, category, description, amount, expense_date, receipt_path)
reimbursement_id → reimbursement

4.3 Mapping decisions

R1 · Surrogate integer keys

Decision	department, employee and reimbursement use INT UNSIGNED AUTO_INCREMENT keys.
Alternatives	Natural keys: dept_name, email, or (applicant_id, claim_date, sequence) for claims.
Why	Natural keys can change (an e-mail address after a name change, a department renamed), and every foreign key would then need ON UPDATE CASCADE. Narrow integer keys make smaller indexes and faster joins. The natural keys are still declared UNIQUE, so they remain candidate keys and duplicates are impossible.
Cost	The ids carry no meaning. AUTO_INCREMENT is not transactional, so rolled-back inserts leave gaps. MySQL also forbids AUTO_INCREMENT columns in CHECK constraints, which is why "cannot supervise yourself" is a trigger.

R2 · Specializations use method 1: a relation per superclass and per subclass

Decision	Both hierarchies map to a superclass relation plus one relation per subclass, whose primary key is also a foreign key to the superclass (§6.8.6.1, first method).
Alternatives	Method 2: only subclass relations, each repeating the inherited attributes. Or a single table with a type column and NULLable subclass columns.
Why	EMPLOYEE is overlapping and partial. Method 2 would copy an employee's details into every role table they hold (Rahul twice), and employees with no role would have nowhere to live. A single table would leave bank details and limits NULL for most rows, and no foreign key could say "this must be a manager". REIMBURSEMENT is disjoint and total, so method 2 is possible in principle. But expense_item, REVIEWS and PAYS all refer to "a claim": with three claim tables there would be no single table to reference, and listing all claims would need a UNION. A single table would make travel, medical and business columns NULLable, so "NOT NULL for travel claims" could not be declared simply.
Cost	Joins to assemble a full claim or employee; totality checked by triggers.

R3 · Many-to-one relationships merge into the "many" side

Decision	WORKS_IN and SUBMITS (total) become NOT NULL foreign keys. SUPERVISES, REVIEWS and PAYS (partial) become nullable foreign keys, and REVIEWS and PAYS bring their descriptive attributes with them (§6.7.6).
Alternatives	A separate relation for each relationship, for example review (reimbursement_id, manager_id, review_date, remarks) and payment (reimbursement_id, admin_id, payment_date, payment_ref).
Why	This is the textbook's combination rule, and it saves joins. The decisive reason: with the review and payment in the same row as the claim, a single-row CHECK can tie them to the status (C5) and enforce separation of duties (C6). If they were in separate tables, every one of those rules would need a trigger.
Cost	A new claim has six NULL columns until it is reviewed and paid. The lifecycle CHECK states exactly when each may be NULL, so the NULLs never mean "unknown"; they mean "has not happened yet".

R4 · No relation for the identifying relationship CONTAINS

Decision	CONTAINS is represented only by expense_item's foreign key.
Why	Its schema would contain exactly the key of expense_item and add nothing (§6.7.5).

R5 · No relation for the aggregation CLAIM_PROCESS

Decision	The aggregation's key is the key of SUBMITS, which (SUBMITS being many-to-one, merged into reimbursement) is reimbursement_id. REVIEWS and PAYS therefore merge into reimbursement.
Why	Textbook §6.8.6.2: no separate relation is needed for an aggregation; the relation of its defining relationship stands for it.

R6 · The multivalued attribute becomes its own relation

Decision	employee_phone (employee_id, phone), both attributes forming the key.
Why	Standard rule for multivalued attributes (§6.7.2), and the only way to keep 1NF.

R7 · One definition of the derived total

Decision	claim_total(id) (a stored function) is used by the approval-limit trigger; the view reimbursement_summary computes totals for lists with one GROUP BY.
Why	The derived attribute has a single definition in the database, so the trigger, the reports and the screens cannot disagree about a claim's total.

R8 · Disjointness of the claim types is checked by a trigger

Decision	Before a row is inserted into travel_, medical_ or business_reimbursement, a trigger checks that the claim's claim_type matches. The subclass primary key allows at most one row per claim in each table, and claim_type can never change.

Alternatives	A purely declarative design: each subclass table gets a constant claim_type column (for example GENERATED ALWAYS AS ('TRAVEL')), and a composite foreign key (reimbursement_id, claim_type) references a UNIQUE (reimbursement_id, claim_type) on reimbursement. Then a TRAVEL row can only reference a TRAVEL claim.
Why	We implemented the declarative version first (MySQL 8 accepts a foreign key on a stored generated column) and removed it during the normalization review. In each subclass table the constant column makes $\varnothing \rightarrow$ claim_type hold: every row has the same value. $\varnothing$ is not a superkey and claim_type is not part of any candidate key, so the table would violate BCNF and 3NF, and it would store "TRAVEL" in every row. Keeping every relation in BCNF mattered more to us than avoiding one trigger.
Cost	The rule is procedural (a trigger) instead of declarative.
Enforced by	`assert_claim_type()` called by `trg_travel_bi`, `trg_medical_bi`, `trg_business_bi`; `trg_reimbursement_bu` (claim_type is immutable).

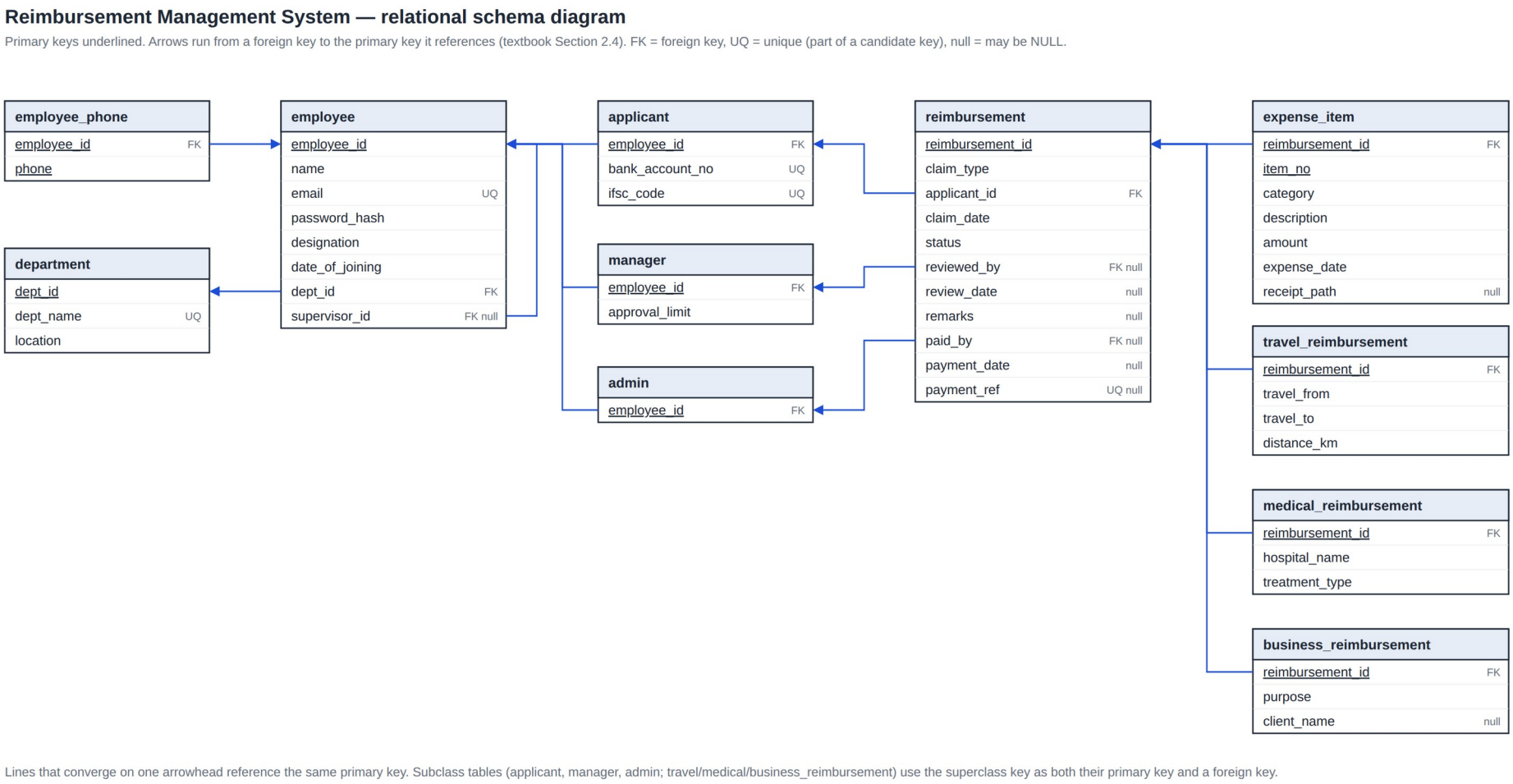


Figure 2. Relational schema diagram: foreign keys point to the primary keys they reference.

5. Functional dependencies and normalization

5.1 Dependencies that hold

Relation	Functional dependencies that must hold (F)	Candidate keys	Highest normal form
department	dept_id → dept_name, location dept_name → dept_id	{dept_id}, {dept_name}	BCNF (so also 3NF)
employee	employee_id → name, email, password_hash, designation, date_of_joining, dept_id, supervisor_id email → employee_id	{employee_id}, {email}	BCNF (so also 3NF)
employee_phone	Only trivial FDs (both attributes form the key)	{employee_id, phone}	BCNF (so also 3NF)
applicant	employee_id → bank_account_no, ifsc_code ifsc_code, bank_account_no → employee_id	{employee_id}, {ifsc_code, bank_account_no}	BCNF (so also 3NF)
manager	employee_id → approval_limit	{employee_id}	BCNF (so also 3NF)
admin	Only trivial FDs	{employee_id}	BCNF (so also 3NF)
reimbursement	reimbursement_id → claim_type, applicant_id, claim_date, status, reviewed_by, review_date, remarks, paid_by, payment_date, payment_ref payment_ref → reimbursement_id (for paid claims; UNIQUE, NULL until paid)	{reimbursement_id}	BCNF (so also 3NF)
travel_reimbursement	reimbursement_id → travel_from, travel_to, distance_km	{reimbursement_id}	BCNF (so also 3NF)
medical_reimbursement	reimbursement_id → hospital_name, treatment_type	{reimbursement_id}	BCNF (so also 3NF)
business_reimbursement	reimbursement_id → purpose, client_name	{reimbursement_id}	BCNF (so also 3NF)
expense_item	reimbursement_id, item_no → category, description, amount, expense_date, receipt_path	{reimbursement_id, item_no}	BCNF (so also 3NF)

5.2 Dependencies that do not hold

A design is only as good as its FDs. Each of these was considered and rejected on the meaning of the data; assuming any of them would change the keys or the normal form.

Dependency that does NOT hold	Reason (real-world fact)	Consequence for the design
name → employee_id	Two employees can share a name.	name is not a key; login uses email.
location → dept_id	Several departments are in Pune.	location is an ordinary attribute of department.
applicant_id → reimbursement_id	An applicant files many claims.	SUBMITS is one-to-many.
applicant_id, claim_date → reimbursement_id	Two claims may be filed on the same day.	No unique constraint on that pair.

Dependency that does NOT hold	Reason (real-world fact)	Consequence for the design
reviewed_by → review_date	A manager reviews claims on many days.	review_date belongs to the review, i.e. the claim.
travel_from, travel_to → distance_km	Distance is what was actually travelled on this trip (detours, return legs).	travel_reimbursement stays in BCNF; see decision E18.
designation → approval_limit	Limits are set per manager, not per job title.	manager stays in BCNF; see decision E17.
hospital_name → treatment_type	A hospital offers many treatments.	Both are attributes of the medical claim.

5.3 Dependencies designed out

Some attributes were deliberately left out because storing them would have introduced a dependency on a non-key attribute:

Attribute we do not store	Dependency it would add	Where it would belong
Bank name and branch	ifsc_code → bank_name, branch (ifsc_code is not a key of applicant)	bank_branch (ifsc_code, bank_name, branch)
Department location in employee	dept_id → location	department (already)
Reviewer's or payer's name in reimbursement	reviewed_by → reviewer_name	employee; the view joins it in
Claim total	Derived from the items (not an FD, but the same kind of redundancy)	Computed by claim_total() and the view
Approval limit by designation	designation → approval_limit	A DESIGNATION relation, if policy changed (E17)
Fixed route distances	travel_from, travel_to → distance_km	A ROUTE relation, if policy changed (E18)

5.4 Normal form of every relation

Applying the definitions to every final relation:

- **1NF** holds everywhere: every column is a single atomic value. The one multivalued attribute (phone) has its own relation.
- **2NF** only matters for relations with a composite key: employee_phone and expense_item. employee_phone has no non-prime attributes. In expense_item no attribute depends on reimbursement_id alone (claim-level facts live in reimbursement) or on item_no alone (item numbers repeat across claims).
- **3NF and BCNF.** In every relation, the left side of every non-trivial FD in the table above is a candidate key (dept_name, email, (ifsc_code, bank_account_no) and payment_ref are keys too). Every relation is therefore in BCNF, which implies 3NF.

5.5 Walkthrough from the unnormalized claim form

To show that the schema is normalized (and not merely normal by luck), we start from the single flat relation a paper claim form or a spreadsheet would produce, and normalize it step by step.

```
claim_form (reimbursement_id, claim_type, claim_date, status,
            applicant_id, applicant_name, email, dept_id, dept_name, location,
            bank_account_no, ifsc_code, phones,
            reviewed_by, approval_limit, review_date, remarks,
            item_no, category, description, amount, expense_date)
```

Functional dependencies (F)

```
F1  reimbursement_id         -> claim_type, claim_date, status, applicant_id,
                                reviewed_by, review_date, remarks
F2  applicant_id             -> applicant_name, email, dept_id, bank_account_no, ifsc_code
F3  email                    -> applicant_id
F4  ifsc_code, bank_account_no -> applicant_id
F5  dept_id                  -> dept_name, location
F6  dept_name                -> dept_id
F7  reviewed_by              -> approval_limit
F8  reimbursement_id, item_no -> category, description, amount, expense_date
```

Candidate key. (reimbursement_id, item_no)$^+$ = all attributes: F8 gives the item columns, F1 the claim columns, then F2, F5 and F7 the rest. Neither attribute appears on the right of any FD, so every key must contain both: it is the only candidate key. Prime attributes: reimbursement_id, item_no.

Step 1: 1NF. `phones` holds a list (e.g. "+9198…, 020…"), which is not atomic. Move it to its own relation. (Repeating each row once per phone would instead create the multivalued dependency applicant_id ↠ phone and multiply every item row.)

```
employee_phone (applicant_id, phone)        key: (applicant_id, phone)
```

Step 2: 2NF. F1 is a partial dependency: claim_type, claim_date, status, applicant_id, … depend on reimbursement_id alone, a proper part of the key (and the applicant, department and reviewer columns follow from them). Split the item-level and claim-level facts:

```
expense_item (reimbursement_id, item_no, category, description, amount, expense_date)
claim        (reimbursement_id, claim_type, claim_date, status, applicant_id, applicant_name,
              email, dept_id, dept_name, location, bank_account_no, ifsc_code,
              reviewed_by, approval_limit, review_date, remarks)
```

Lossless: expense_item ∩ claim = {reimbursement_id}, which is a key of claim (§7.2.3 test). expense_item is now in BCNF: its only non-trivial FD is F8, whose left side is its key.

Step 3: 3NF. claim (key reimbursement_id) still has transitive dependencies: reimbursement_id → applicant_id → applicant_name, email, …; applicant_id → dept_id → dept_name, location; and reimbursement_id → reviewed_by → approval_limit. Each makes a non-prime attribute depend on a non-key attribute. Decompose on F2, F5 and F7:

```
reimbursement     (reimbursement_id, claim_type, claim_date, status, applicant_id,
                   reviewed_by, review_date, remarks)
applicant_details (applicant_id, applicant_name, email, dept_id, bank_account_no, ifsc_code)
department        (dept_id, dept_name, location)
manager           (reviewed_by -> employee_id, approval_limit)
```

Lossless: each split shares an attribute that is a key of one side (applicant_id of applicant_details, dept_id of department, reviewed_by of manager).

Dependency preservation. Every FD lies inside one final relation: F1 in reimbursement; F2, F3, F4 in applicant_details; F5, F6 in department; F7 in manager; F8 in expense_item. So each can be checked without a join (§7.4.4).

Cross-check with the 3NF synthesis algorithm (§7.5.2). F is already a canonical cover (no extraneous attributes, one FD per left side). One schema per FD gives the relations above (F3, F4 and F6 fall inside F2's and F5's schemas and are dropped), and the candidate key (reimbursement_id, item_no) is contained in expense_item, so no extra key schema is needed. The algorithm produces the same result.

Result

The final relations are exactly the schema we derived from the ER diagram, with one difference: normalization alone keeps applicant_details as one relation. Our design splits it into **employee** and **applicant** because of the partial specialization: employees who are not applicants (an intern, a finance clerk) would otherwise need NULL bank details. Both parts are in BCNF.

The payment columns (paid_by, payment_date, payment_ref) and the claim-type details were left out above to keep it short. They behave the same way: payment columns depend on reimbursement_id only, and the type-specific details move to the subclass relations to avoid NULLs, not because of any normal form.

6. Constraint and implementation decisions

C1 · Domains with CHECK on VARCHAR, not ENUM

Decision	claim_type, status, category and treatment_type are VARCHAR columns with named CHECK (… IN (…)) constraints.
Alternatives	MySQL ENUM columns; lookup tables.
Why	CHECK is standard SQL (§4.4.4) and works the same in other systems; ENUM is MySQL-only. ENUM sorts by position rather than by value and, outside strict mode, silently stores an empty string for an invalid value. Named constraints let the server turn a violation into a readable message. Lookup tables are the better choice once the values have attributes of their own (see E14).
Note	MySQL enforces CHECK constraints only from version 8.0.16. Earlier versions parsed them and silently ignored them, so the project requires MySQL 8.0.16 or later.

C2 · Money is DECIMAL, never FLOAT

Decision	amount DECIMAL(10,2); approval_limit DECIMAL(12,2).
Why	FLOAT and DOUBLE are binary fractions: 0.10 has no exact representation, and sums drift by paise. DECIMAL stores exact decimal digits, which is what money needs.

C3 · Number-like identifiers are strings

Decision	bank_account_no, phone and ifsc_code are VARCHAR/CHAR with pattern CHECKs.
Why	Nobody does arithmetic on them; leading zeros matter (002101567890), phone numbers may start with +, and an 18-digit account number would overflow INT. CHECKs with regular expressions still guarantee digits only.

C4 · Referential actions, chosen per foreign key

Foreign key	ON DELETE	Why
employee_phone, applicant, manager, admin → employee	CASCADE	They are parts of the same person (multivalued attribute, subclass rows). They only cascade if nothing else blocks the delete.
travel_/medical_/business_reimbursement, expense_item → reimbursement	CASCADE	Existence dependency: withdrawing a claim removes its details and items.

Foreign key	ON DELETE	Why
employee.dept_id → department	RESTRICT	A department with staff cannot be deleted; move the employees first.
employee.supervisor_id → employee	RESTRICT	Reassign subordinates before deleting their supervisor.
reimbursement.applicant_id → applicant	RESTRICT	Financial records are kept: an employee with claims cannot be deleted or lose the applicant role.
reimbursement.reviewed_by → manager; paid_by → admin	RESTRICT	Audit trail: someone who reviewed or paid claims keeps that role on record.

ON UPDATE is the default (RESTRICT) everywhere: surrogate keys never change. MySQL also forbids referential actions on columns used in CHECK constraints; applicant_id, reviewed_by and paid_by appear in CHECKs, so RESTRICT is both what we want and what MySQL requires there.

C5 · Status and the review and payment columns must agree

Decision	`chk_reimbursement_lifecycle` lists, for each status, which review and payment columns must be set and which must be NULL (for example, REJECTED needs remarks; PAID needs paid_by, payment_date and payment_ref).
Why	It makes contradictory rows impossible, such as a SUBMITTED claim with a payment reference or a PAID claim with no reviewer. This is what makes the controlled redundancy of E11 safe.

C6 · Separation of duties as single-row CHECKs

Decision	reviewed_by ≠ applicant_id; paid_by ≠ applicant_id; paid_by ≠ reviewed_by.
Why	Nobody approves or pays their own claim, and no single person can both approve and pay a claim. Because REVIEWS and PAYS were merged into reimbursement (R3), these are row-level rules, so a CHECK suffices.

C7 · Triggers: catalogue

Triggers are used only where no declarative constraint can express the rule. Two helper procedures (`assert_claim_editable`, `assert_claim_type`) keep the shared checks in one place.

Trigger	Fires	Rule	Why not a CHECK
trg_employee_bi	BEFORE INSERT employee	An employee cannot supervise themselves	employee_id is AUTO_INCREMENT, which MySQL forbids in CHECKs
trg_employee_bu	BEFORE UPDATE employee	No cycles in SUPERVISES (recursive query)	Reads other rows
trg_reimbursement_bi	BEFORE INSERT reimbursement	Starts as SUBMITTED; claim_date not in the future	Uses CURRENT_DATE; a rule about the starting state
trg_reimbursement_bu	BEFORE UPDATE reimbursement	Identity fixed; legal transitions; items and details exist before review; total ≤ reviewer's limit; dates not in the future; the review cannot change when paying	Compares OLD and NEW; reads expense_item, manager and the subclass tables
trg_reimbursement_bd	BEFORE DELETE reimbursement	Only unreviewed claims can be withdrawn	CHECKs do not apply to deletes

Trigger	Fires	Rule	Why not a CHECK
trg_expense_item_bi / _bu	BEFORE INSERT / UPDATE expense_item	Claim still editable; expense_date ≤ claim_date; an item cannot move or be renumbered	Reads reimbursement
trg_expense_item_bd	BEFORE DELETE expense_item	Claim still editable; at least one item remains	Counts other rows
trg_travel_bi, trg_medical_bi, trg_business_bi	BEFORE INSERT	Details match the claim type (disjoint); claim still editable	Reads reimbursement
trg_travel_bu, trg_medical_bu, trg_business_bu	BEFORE UPDATE	Claim still editable; details cannot move to another claim	Reads reimbursement
trg_travel_bd, trg_medical_bd, trg_business_bd	BEFORE DELETE	Details are removed only with their claim (keeps the specialization total)	CHECKs do not apply to deletes

C8 · Total participation is checked at the review gate

Decision	Before a claim can be reviewed, the trigger checks that it has at least one expense item and a details row for its type.
Why	A claim is inserted first and its items afterwards, so "at least one item" cannot be checked when the claim row is inserted. It would need a deferred constraint (checked at COMMIT), which MySQL does not support (§4.4.7). The application inserts claim, details and items in one transaction, so a claim never appears without them. The review gate is the database's own guarantee: an incomplete claim (from a bug or a manual insert) can never be approved, rejected or paid. Direct deletes of the last item, or of the details row, are refused. Deleting the whole claim still works because MySQL does not fire triggers for cascaded foreign-key deletes.

C9 · Reviewed claims are frozen

Decision	Once a claim leaves SUBMITTED, its items and type-specific details cannot be inserted, changed or deleted, and the claim cannot be withdrawn.
Why	Otherwise an applicant could raise the amounts after approval, and the paid total would differ from the approved total (it would also bypass the approval-limit check).

C10 · The supervision hierarchy stays a tree

Decision	When supervisor_id changes, a trigger walks up the chain from the new supervisor with a recursive query (§5.4); reaching the employee means a cycle, and the update is refused.
Why	A foreign key guarantees the supervisor exists, not that the chain ends. A cycle would make "who is above whom" meaningless and would make recursive reports loop.

C11 · Indexes

Decision	Primary keys and UNIQUE constraints are indexed automatically, and InnoDB indexes every foreign key. Two extra indexes: reimbursement(status) for the review and payment queues, and reimbursement(claim_date) for monthly reports.

Why	Those two columns filter the most frequent queries. More indexes would slow every insert and update for little gain at this data size.

C12 · InnoDB and utf8mb4

Decision	Every table uses the InnoDB engine; the database uses the utf8mb4 character set.
Why	InnoDB is the MySQL engine that enforces foreign keys and supports transactions and row-level locking; MyISAM silently ignores foreign keys. utf8mb4 stores any Unicode text, including names in Indian scripts.

C13 · The application uses a least-privilege database account

Decision	The server connects as rms_app, which may SELECT, INSERT, UPDATE, DELETE and EXECUTE on rms only. The schema is created by root.
Why	A bug or an injected query in the application cannot drop a table or disable a trigger or constraint (§4.7, authorization).
Side effect	Without table-level privileges, MySQL reports a blocked delete as error 1217 without naming the constraint (instead of 1451), so the server supplies its own message in those places.

C14 · Transactions and concurrency in the application

Decision	Writes that span tables run in one transaction (submitting a claim; creating or editing an employee with phones and roles). Review and payment updates include the expected status in the WHERE clause.
Why	Atomicity: if the third item of a claim violates a CHECK, the claim and its first two items disappear too, and the uploaded files are deleted. Two managers deciding the same claim at once: InnoDB locks the row for the first UPDATE; the second runs after it, no longer matches status = 'SUBMITTED', updates no rows, and gets "already reviewed". No decision is overwritten.

C15 · Date rules that need today's date are triggers

Decision	claim_date, review_date and payment_date may not be in the future; expense_date may not be after claim_date.
Why	MySQL forbids non-deterministic functions such as CURRENT_DATE in CHECK constraints, and expense_date and claim_date are in different tables. Rules that only compare columns of one row (review_date ≥ claim_date, payment_date ≥ review_date) remain CHECKs.

6.1 Catalogue of named constraints

Constraint	Table	Rule
chk_employee_email	employee	email looks like an address
chk_phone_format	employee_phone	10 to 14 digits, optional leading +

Constraint	Table	Rule
chk_applicant_account / chk_applicant_ifsc	applicant	9–18 digits / 4 capital letters, 0, 6 letters or digits
chk_manager_limit	manager	approval_limit > 0
chk_reimbursement_type / _status	reimbursement	Fixed sets of values
chk_reimbursement_lifecycle	reimbursement	Review and payment columns match the status (C5)
chk_reimbursement_no_self_review / _no_self_payment / _payer_not_reviewer	reimbursement	Separation of duties (C6)
chk_reimbursement_review_after_claim / _payment_after_review	reimbursement	Dates in order
chk_travel_distance	travel_reimbursement	distance_km > 0
chk_medical_treatment	medical_reimbursement	Fixed set of treatment types
chk_item_no / chk_item_amount / chk_item_category	expense_item	item_no ≥ 1; amount > 0; fixed set of categories
uq_department_name, uq_employee_email, uq_applicant_bank, uq_reimbursement_payment_ref	various	Candidate keys (UNIQUE)

7. Testing

db/06_constraint_tests.sql is a test suite that runs inside the database. Each test calls expect_error (the statement must be rejected) or expect_ok (it must succeed), and the whole run is rolled back, so the data is unchanged afterwards. The 50 tests cover:

- Domain CHECKs: negative amount, unknown category, status or treatment type, malformed IFSC or phone number.
- Keys: duplicate e-mail, bank account, item number or payment reference; a second details row for a claim.
- Foreign keys: a non-applicant filing a claim, a non-manager reviewing, deleting a staffed department or an employee with history.
- Separation of duties and lifecycle: self-review, self-payment, approver paying, rejection without remarks, approval without a reviewer.
- Triggers: approval limit, illegal transitions, closed claims, future dates, frozen items, the last item, disjointness, supervision cycles, and the totality gates.
- Cascades: withdrawing a claim leaves no orphan rows.

8. Known limitations

Stating these in the viva shows you know where the design stops:

Limitation	Why it exists	What we would do next
"At least one item" and "exactly one details row" are not checked at insert time	MySQL has no deferred constraints	Checked at the review gate and by transactions; PostgreSQL could use deferrable constraint triggers

Limitation	Why it exists	What we would do next
No history of status changes or approval limits	Out of scope; the row keeps who and when for each step	An append-only claim_event (claim, status, by, at) table filled by triggers
Withdrawn claims are deleted	Simpler lifecycle	A CANCELLED status instead of DELETE
Single-level approval only	Kept REVIEWS one-to-many	Make REVIEWS many-to-many with an approval level
Receipt files are served to anyone who knows the random file name	Simplicity of a lab project	Serve files through an authenticated route
AUTO_INCREMENT values are not reused after a rollback	InnoDB design	Harmless; ids carry no meaning
One currency (INR), one payout account per employee	Scope	currency column; a BANK_ACCOUNT entity

9. Viva preparation

Likely questions and short answers. Each answer names the decision behind it, so you can read further above.

9.1 ER modelling

Q. Why is DEPARTMENT an entity set and not an attribute?

It has an attribute of its own (location) and is shared by many employees. As an attribute it would repeat in every employee row and create employee_id → department → location, a transitive dependency. Textbook §6.9.2. (E1)

Q. What does the double line between EMPLOYEE and WORKS_IN mean? Where does the arrow point?

Total participation: every employee works in a department. The arrow points to DEPARTMENT, the "one" side: each employee works in at most one department. Together with the double line: exactly one (1..1).

Q. On SUBMITS, which side does 0..* describe?

The APPLICANT side: an applicant submits zero or more claims. 1..1 next to REIMBURSEMENT says each claim is submitted by exactly one applicant. The label describes the entity set it sits next to; reading it the other way round is the mistake the textbook warns about in §6.4.

Q. Why is SUPERVISES on EMPLOYEE and not on MANAGER?

Reporting lines and approval authority are different. Sneha supervises Meera but is not a manager. Role indicators supervisor and subordinate say which end is which. (E3)

Q. Explain overlapping/disjoint and partial/total with your two specializations.

EMPLOYEE: overlapping (Sneha is applicant and admin) and partial (Karan is in no subclass); drawn with three separate arrows and no "total". REIMBURSEMENT: disjoint (each claim is one type) and total (every claim has a type); one shared arrow and the keyword "total". (E4, E10)

Q. ADMIN has no attributes. Why is it a subclass?

Because it takes part in a relationship the superclass does not: PAYS. Textbook §6.8.1 allows a subclass for either reason. (E5)

Q. Why aggregation? Why not a ternary relationship?

A ternary REVIEWS (manager, applicant, reimbursement) would record the applicant again and allow a review whose applicant differs from the submitter (§6.8.5). Honestly, a binary REVIEWS to REIMBURSEMENT would give the same tables here because SUBMITS is many-to-one; we keep the aggregation because a manager decides a submission (who claimed what), and the self-review and payee rules involve both. (E7)

Q. Why rename APPROVES to REVIEWS?

A manager may reject a claim; one relationship records either decision, and the status holds the outcome. (E8)

Q. What is a weak entity set? Why is EXPENSE_ITEM weak?

One without a key of its own: it is identified by its owner's key plus a discriminator. An item is "item 2 of claim 17"; it cannot exist without its claim. CONTAINS is the identifying relationship (double diamond), and item_no is the discriminator (dashed underline). (E13)

Q. Why does REIMBURSEMENT have a double line to CONTAINS too?

A claim must have at least one item (1..*). The weak entity side is total by definition; the claim side is total by our business rule.

Q. Why is total_amount not stored?

It is derived from the items; storing it creates an update anomaly risk on every item change. It is computed by a function and a view. (E12)

9.2 From ER to tables

Q. How did you map the ISA hierarchies? Why not the second method or one big table?

Method 1: superclass table plus subclass tables keyed by the superclass key. Method 2 fails for overlapping/partial EMPLOYEE (copies, and no home for employees without a role) and would leave nothing for expense_item to reference for REIMBURSEMENT. A single table means NULL-filled columns and no way to point a foreign key at "managers only". (R2)

Q. Why is there no table for REVIEWS, PAYS, SUBMITS, WORKS_IN, SUPERVISES, CONTAINS or CLAIM_PROCESS?

Many-to-one relationships merge into the many side (§6.7.6); CONTAINS is redundant (§6.7.5); an aggregation needs no relation (§6.8.6.2). (R3–R5)

Q. Merging REVIEWS gives NULLs. Why accept them?

The textbook allows merging with NULLs for partial participation. It lets single-row CHECKs tie the review to the status and enforce separation of duties. The NULLs mean "not yet happened", and the lifecycle CHECK says exactly when each may be NULL. (R3, C5)

Q. Why surrogate keys?

Natural keys such as e-mail can change; integer keys are small and stable. The natural keys are still UNIQUE, so they remain candidate keys. (R1)

Q. How do you ensure a claim has exactly one details row of the right type?

Primary key on each details table (at most one per table); a trigger checks the type matches (disjoint); claim_type never changes; the review gate refuses a claim with no details row (total). (R8, C8)

Q. Why not a composite foreign key with a constant claim_type column for disjointness?

The constant column makes $\varnothing \rightarrow$ claim_type hold in the subclass table. $\varnothing$ is not a superkey and claim_type is not prime, so the table would break BCNF and 3NF. We tried it and removed it. (R8)

Q. Explain your ON DELETE choices.

CASCADE where the child is part of the parent (phones, subclass rows, claim details, items). RESTRICT where history must be kept (claims, reviews, payments) or data must be moved first (departments, supervisors).

9.3 Functional dependencies and normalization

Q. Define 3NF and BCNF. Which is each of your relations in?

BCNF: every non-trivial FD has a superkey on the left. 3NF additionally allows $\alpha \rightarrow \beta$ when every attribute of $\beta - \alpha$ is prime. All 11 relations are in BCNF, hence in 3NF.

Q. List the FDs of reimbursement.

reimbursement_id → all other attributes; payment_ref → reimbursement_id for paid claims (UNIQUE). Nothing else: applicant_id, reviewed_by and status do not determine other columns.

Q. Is (travel_from, travel_to) → distance_km an FD?

No: distance is what was travelled on this trip. If the policy fixed distances per route it would be, and we would need a ROUTE relation to stay in 3NF. (E18)

Q. Is status redundant with paid_by?

Partly: PAID exactly when paid_by is not NULL. That is a rule across columns of one row (enforced by a CHECK), not a functional dependency, so it does not affect the normal form; and APPROVED versus REJECTED is not derivable from anything else. (E11)

Q. Walk through 1NF → 2NF → 3NF.

Section 7.3: phones out for 1NF; split item-level from claim-level facts for 2NF (partial dependency on reimbursement_id); split applicant, department and manager facts for 3NF (transitive dependencies).

Q. Prove one of your decompositions is lossless.

expense_item ∩ claim = {reimbursement_id}, and reimbursement_id → all attributes of claim, so it is a superkey of claim: lossless by the test in §7.2.3.

Q. Is the decomposition dependency-preserving?

Yes: every FD F1–F8 lies entirely inside one final relation, so each can be checked locally.

Q. Does employee_phone satisfy 4NF?

Yes. The multivalued dependency employee_id ↠ phone is trivial in a relation that has only those two attributes.

Q. If approval limits depended on designation, what would change?

designation → approval_limit would hold. Stored in manager or employee it would be a transitive dependency; we would add DESIGNATION (designation, approval_limit). (E17)

9.4 Constraints and implementation

Q. Why CHECK instead of ENUM?

Standard SQL, named constraints, no silent empty strings. MySQL enforces CHECK only from 8.0.16. (C1)

Q. Why DECIMAL for money?

Binary floating point cannot represent 0.10 exactly; DECIMAL is exact. (C2)

Q. Why triggers? Which rules could not be CHECKs?

Anything that reads another row or table (approval limit, at least one item, cycles), compares OLD with NEW (transitions), or needs CURRENT_DATE. SQL assertions would express some of them, but MySQL (like most systems) does not support assertions (§4.4.8).

Q. Two managers approve the same claim at the same moment. What happens?

InnoDB locks the row for the first UPDATE. The second waits, then finds status is no longer SUBMITTED, matches no row, and reports "already reviewed". (C14)

Q. The server crashes halfway through saving a claim. What happens?

The claim, its details and its items are in one transaction, so none of them is committed. (C14)

Q. A manager's limit is lowered after they approved a large claim. Is the approval now invalid?

No. The limit is checked when the decision is made; past decisions are historical facts. A limit history table would be needed to show what the limit was on a given day.

Q. How do you prevent SQL injection?

Every value is passed as a ? parameter, never concatenated. The only names built into SQL come from fixed lists in the code. And the application's database account cannot change the schema. (C13)

Q. Why can't an employee be deleted?

If they have claims, reviews, payments or subordinates, RESTRICT foreign keys refuse it: financial records are kept.

Q. What are the weaknesses of your design?

See Section 8: no deferred constraints, no history of statuses or limits, withdrawn claims leave no trace, receipt files are not access-controlled, and single-level approval only.

10. Demonstration script

Commands to rebuild everything from scratch and show the rules working:

```
docker compose up -d                                    # MySQL 8.0 on port 3307 (or use a local
MySQL)
./db/setup.sh                                           # schema, views, triggers, app user, sample
data
mysql -h127.0.0.1 -P3307 -uroot -p --protocol=TCP < db/06_constraint_tests.sql   # 50 tests
cd server && cp .env.example .env && npm install && npm start                    # API on :4000
cd client && npm install && npm run dev                                          # UI on :5173
```

A suggested live demo, in this order:

1. Sign in as Vikram (applicant) and submit a travel claim with two items: one transaction writes reimbursement, travel_reimbursement and expense_item.
2. Sign in as Neha (manager, limit ₹10,000) and try to approve Priya's ₹30,000 claim (#24): the trigger refuses, and its message appears on screen.
3. Sign in as Rahul (manager, limit ₹50,000), approve Vikram's claim, then try to withdraw it as Vikram: refused, because it has been reviewed.
4. Sign in as Sneha (admin) and pay it with a UTR reference; try the same reference on another claim: refused, because payment_ref is unique.
5. In Employees, try to remove Rahul's manager role: refused, because he has reviewed claims (audit trail).
6. Run db/06_constraint_tests.sql to show all 50 tests passing, and db/07_report_queries.sql for the recursive and window-function queries.